

Testing Strategy

Risk-based quality assurance for the KiwiShare mobile marketplace

VERSION	COURSE	OWNER
0.1 - Draft for team review	COMPSCI 734	Five Guys

Prepared from COMPSCI 734 Lecture 07, the approved KiwiShare pitch, product requirements, and module backlog.

Strategy at a glance

A sustainable testing system for a five-person team: fast feedback for every change, deeper confidence at integration points, and risk-weighted assurance before release.

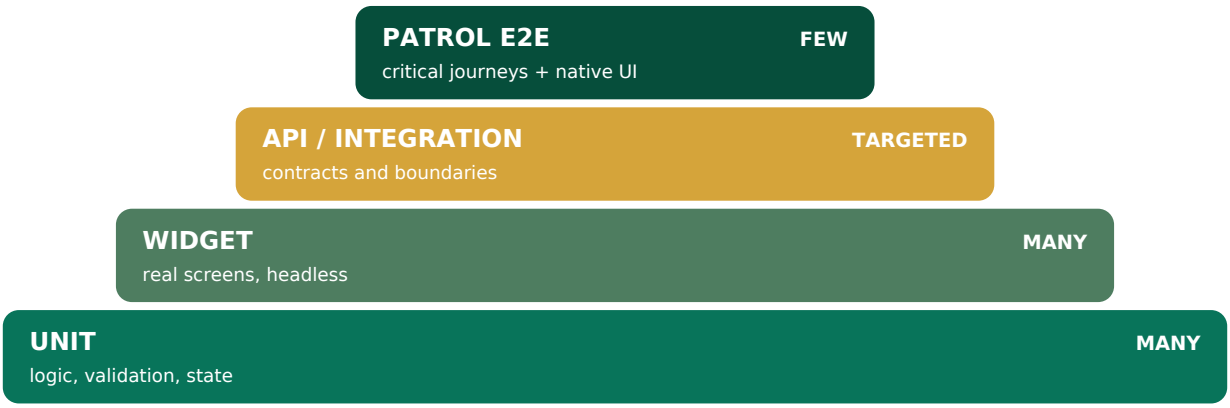
01 SHIFT LEFT
Catch logic and UI defects in fast tests before they reach shared branches.

02 TEST THE STATES
Success is not enough: loading, empty, error, offline and denied states matter.

03 PROTECT TRUST
Payment, QR, authorization, location, chat and reporting receive highest assurance.

04 GATE THE FLOW
Task branch to pre to main: quality evidence travels with every change.

Fast feedback at the base; highest confidence and cost at the top



Many unit tests + many widget tests + targeted integration tests + a few critical Patrol journeys.

Contents

1. Purpose
2. Source and related documents
3. Quality objectives
4. Scope
5. Testing principles
6. Risk-based prioritisation
7. Test levels
8. Critical journey coverage matrix
9. State and failure coverage
10. Test environments and data
11. Third-party and native capability strategy
12. Non-functional testing
13. Device and platform coverage
14. Automation and branch quality gates
15. Coverage and quality targets
16. Defect classification
17. Entry and exit criteria
18. Traceability, naming, and evidence
19. Ownership and maintenance
20. Decisions required before CI enforcement
21. Initial implementation sequence
Appendices A-C

Reading route: Product and engineering stakeholders should begin with Sections 3, 6, 8, 14 and 17. Contributors implementing tests should also read Sections 5, 7, 9, 10 and 11.

Field	Value
Document owner	Five Guys
Product	KiwiShare
Course	COMPSCI 734 - Mobile, Web & Enterprise Computing
Version	0.1 (Draft for team review)
Status	Proposed
Last updated	15 August 2026
Primary branch	`pre`

1. Purpose

This document defines how the KiwiShare team will verify product quality throughout development and before release. It converts the product risks, system architecture, and course guidance into a repeatable testing process that five contributors can sustain.

The strategy is intended to:

- detect defects as early and cheaply as possible;
- protect the critical marketplace and transaction journeys;
- verify loading, success, empty, error, offline, and permission-denied states;
- provide fast feedback on every change and stronger assurance before merge or release;
- make quality evidence traceable to product requirements and GitHub issues;
- reduce reliance on slow, inconsistent manual regression testing.

This is a strategy document, not a catalogue of individual test cases. Detailed cases belong in the test code, issue acceptance criteria, and release/UAT checklists.

2. Source and Related Documents

This strategy is based on:

- `CS734 Lecture 07 - Testing Mobile Applications`;
- `COMPSCI 734_Five Guys_KiwiShare Elevator Pitch - Assignment 1`;
- the approved KiwiShare product requirements;
- the current KiwiShare module backlog and issue decomposition.

It should be read together with:

- `docs/product-requirements/product-requirements.md`;
- `docs/architecture.md`;
- `docs/api-spec.md`;
- `docs/schemas/`;
- `docs/security-owasp.md`;
- `docs/design-guidelines/`;
- `docs/gitlab-flow.md`;
- `docs/deployment.md`.

Where this document conflicts with an approved product requirement, API contract, schema, or security control, the conflict must be raised and resolved before implementation proceeds.

3. Quality Objectives

KiwiShare is considered testable and release-ready only when the team has evidence that it is:

1. **Functionally correct** - required user journeys and business rules behave as specified.
2. **Safe and trustworthy** - identity, authorization, reporting, payment, order verification, and reputation controls cannot be bypassed through normal or adversarial use.
3. **Reliable** - failures in network, backend, permissions, notifications, or third-party services produce controlled and recoverable outcomes.
4. **Accessible and usable** - core screens meet automated tap-target, semantic-label, and text-contrast checks and remain usable under realistic device conditions.
5. **Privacy-preserving** - personal, location, chat, report, and payment-related data are exposed only to authorized actors.
6. **Maintainable** - tests are deterministic, readable, isolated, and fast enough to run routinely.

4. Scope

4.1 In scope

This strategy covers the following product surfaces where included in the implemented MVP:

- Flutter mobile application;
- authentication and profile management;
- product discovery, search, filtering, sorting, and item details;
- item publication, image upload, and AI-assisted descriptions;
- watchlist and price-change notifications;
- contextual chat, text sanitization, image/voice attachments, and push notifications;
- location permission, nearby items, and safe-zone suggestions;
- trade, payment, order, QR verification, completion, cancellation, and refund paths;
- reports, blocking, moderation, ratings, and user credit;
- Firebase-managed services and security rules;
- privileged Node.js services/cloud functions and data persistence;
- React administration features, if delivered within the release scope;
- CI checks, test environments, and release-quality evidence.

4.2 Out of scope for the initial coursework release

Unless separately approved, the following are not mandatory for the first release:

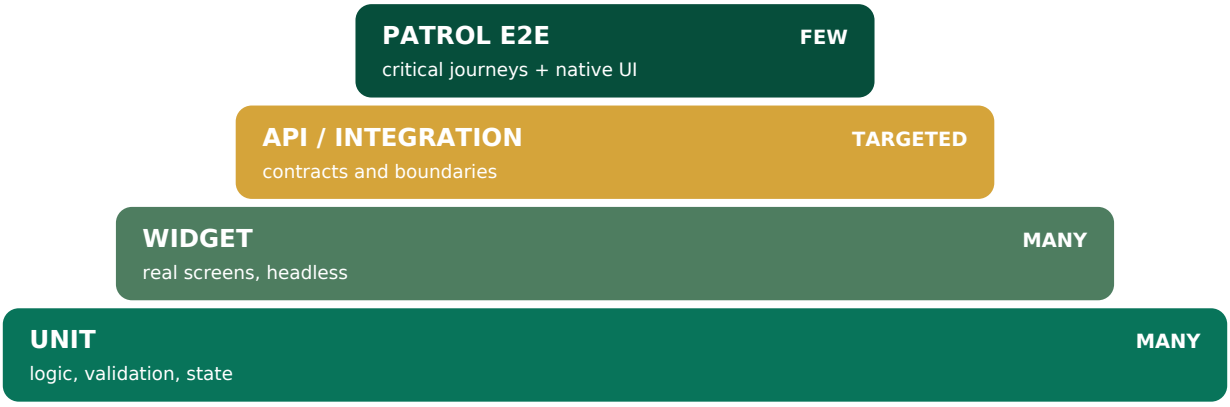
- a large physical-device farm;
- exhaustive testing of every Android and iOS model/version combination;
- full-scale production load testing;
- pixel-perfect golden tests for every screen;

- penetration testing by an external security provider;
- testing the internal correctness of Google, Firebase, Stripe, FCM, map, social-sharing, or AI vendor platforms.

External services are instead covered through contract validation, approved emulators/test modes, controlled fakes, and a small number of end-to-end integration checks.

5. Testing Principles

Fast feedback at the base; highest confidence and cost at the top



The course principle translated into KiwiShare: maximize fast confidence below and reserve expensive native E2E tests for the journeys that matter most.

5.1 Follow the mobile testing pyramid

KiwiShare will maintain:

- **many unit tests** for business logic, validation, state transitions, and view models;
- **many widget/component tests** for screens, interactions, UI states, and accessibility;
- **enough API/integration tests** to verify boundaries between components and services;
- **few end-to-end tests** for the critical journeys that would be unacceptable to fail during a demonstration or release.

The default response to a defect is to add the lowest-level regression test capable of reproducing it reliably.

5.2 Test observable behaviour, not implementation details

Tests should verify inputs, outputs, state, user-visible behaviour, contracts, and authorized effects. They should not become coupled to private methods, arbitrary widget-tree structure, or incidental implementation choices.

5.3 Design for testability

Network clients, repositories, clocks, identifier generators, storage adapters, notification gateways, location providers, payment gateways, and AI services must be injectable. Real network calls do not belong in unit or widget tests.

5.4 Make tests deterministic

Automated tests must not depend on:

- production services or production data;
- a developer manually starting a backend;
- real payment cards or personal accounts;
- uncontrolled system time, random values, or live GPS coordinates;
- execution order or state left by another test.

5.5 Automate repeated checks; use manual testing for judgment

Automation protects stable, repeatable expectations. Manual exploratory testing and UAT remain necessary for usability, visual quality, unclear requirements, and real-user feedback, but they do not replace automated regression tests.

6. Risk-Based Prioritisation

P0	**CRITICAL**	money, authorization, privacy, transaction integrity
P1	**HIGH**	core marketplace and trust journeys
P2	**MEDIUM**	experience degradation with a workaround
P3	**LOW**	cosmetic or low-impact behaviour

Testing effort is allocated according to impact and likelihood rather than equally across all features.

Priority	Meaning	KiwiShare examples	Required assurance
P0 - Critical	Failure could cause financial loss, unauthorized access, privacy exposure, corrupt transaction state, or an unusable core product	authorization, payment, refund, order ownership, QR verification, duplicate requests, private location/chat/report data	unit + boundary integration + E2E happy path and critical failure paths
P1 - High	Failure blocks a primary marketplace journey or materially damages trust	sign-in, publish item, item retrieval, search, chat, report/block, rating/credit, safe-zone permission handling	unit + widget + integration; selected E2E
P2 - Medium	Failure degrades experience but has a workaround	watchlist, push notification, dark mode, social sharing, AI suggestions	unit/widget and targeted integration
P3 - Low	Cosmetic or low-impact behaviour	non-critical animation, decorative layout, app icon	widget/manual visual review as appropriate

No P0 feature may rely only on a manual happy-path demonstration.

7. Test Levels

7.1 Unit tests

Purpose: Verify one function, class, service, reducer, controller, or view model without a real UI, device, network, or external service.

Expected volume: Many.

Flutter/Dart scope:

- field validation and form rules;
- item search/filter/sort logic;
- price and category validation;
- publish-item view-model states;
- permission and location fallback decisions;
- chat sanitization and message-state logic;
- watchlist changes and notification eligibility;
- order and QR state-transition rules;
- report, rating, and credit calculations;
- mapping API/domain errors to safe user messages;
- repository/service calls occur exactly as expected.

Node.js/backend scope:

- request validation and normalization;
- authorization and ownership checks;
- idempotency and duplicate-event protection;
- allowed and rejected state transitions;
- payment/refund orchestration logic using a fake gateway;
- QR generation/verification rules;
- content moderation/sanitization;
- notification eligibility and payload construction;
- data access and error mapping.

React scope, where applicable:

- state and form logic;
- authorization-driven actions;
- moderation and item-management behaviours.

Method:

- use hand-written fakes or `mocktail` at injectable seams in Dart;
- use the project-approved mocking library in Node/React;
- use fixed clocks, deterministic identifiers, and explicit fixtures;
- verify success, invalid input, missing data, unauthorized access, dependency failure, and retry/idempotency paths where relevant.

Location and naming:

- Flutter tests live under `test/` and end in `_test.dart`;
- backend and web tests follow the relevant package convention;
- tests should use descriptive behaviour-based names, for example: `rejects completion when only one party has verified the QR code`.

7.2 Flutter widget tests

Purpose: Render real Flutter widgets headlessly and verify whole-screen behaviour at near-unit-test speed.

Expected volume: Many; every core screen should have widget coverage.

Each core screen should demonstrate, where applicable:

- loading state;
- populated/success state;
- empty state;
- validation state;
- recoverable error state;
- offline/degraded state;
- authorized and unauthorized action state;
- navigation and primary interaction;
- light and dark theme behaviour where required;
- accessibility guidelines.

Examples include:

- sign-in form validation and error rendering;
- product list, no-result state, sort/filter, and navigation to details;
- owner-only edit/delete controls on an item detail screen;
- publish form validation, upload progress, and failure recovery;
- chat list/history, message sending, attachment errors, and report action;
- profile, credit score, published/sold item states, and logout;
- watchlist add/remove and price-change indication;
- payment and QR status screens using fake repositories.

Implementation guidance:

- pump the real app or screen shell with fake repositories/services injected;
- use `find.text`, `find.byType`, `find.byIcon`, and stable `Key` values;
- use `tester.tap`, `enterText`, `drag`, and related interaction APIs;
- use `pump()` when asserting an intermediate/loading frame;
- use `pumpAndSettle()` only when the widget tree is expected to settle; do not use it blindly with indefinite animations;
- prefer semantic labels and stable test keys over brittle widget-tree traversal.

7.3 Accessibility tests

Accessibility checks are automated build gates for core screens, not optional manual reminders.

Widget tests must use semantics and validate, as applicable:

- `androidTapTargetGuideline` - tappable targets are at least 48 x 48 dp;
- `iOSTapTargetGuideline` - tappable targets are at least 44 x 44 pt;
- `textContrastGuideline` - text meets the framework's minimum contrast guidance;
- `labeledTapTargetGuideline` - tappable controls have meaningful semantic labels.

Additional manual checks should cover:

- large text/text scaling;
- screen-reader reading order on critical journeys;
- error messages that do not rely on colour alone;
- light/dark mode readability;
- keyboard focus and navigation for the web administration surface.

7.4 API and service integration tests

Purpose: Verify that components work together across a real boundary while retaining controlled test data.

Expected volume: Targeted; more than E2E, fewer than unit/widget tests.

Coverage should include:

- API request/response schema and error contracts;
- authentication token validation and authorization rules;
- Firebase/Firestore security rules, where used;
- Node.js service-to-database behaviour;
- serialization/deserialization and backward-compatible contracts;
- item creation/update/delete and owner restrictions;
- chat persistence and participant-only access;
- watchlist persistence and notification trigger conditions;
- order creation, payment-event handling, refunds, and idempotency;
- QR verification from both authorized parties;
- report lifecycle and moderation access;
- rating/credit updates only after eligible completed transactions.

Tests must use isolated test databases, Firebase emulators where practical, and vendor-approved sandbox/test modes. Production credentials and production data are prohibited.

7.5 Mobile end-to-end tests with Patrol

Purpose: Exercise the real mobile application on an emulator/device, including native platform UI that Flutter widget tests cannot control.

Expected volume: Few; one focused test per critical user journey.

Flutter's `integration_test` may be used for pure-Flutter flows, but it cannot fully automate native permission dialogs, notifications, and platform views. KiwiShare should use Patrol for journeys that cross

those boundaries.

Required initial Patrol coverage:

1. **Authentication smoke journey** - launch, authenticate with a controlled test identity, verify the signed-in state, and log out.
2. **Publish-item journey** - capture/select an image, enter details, handle permission state, submit, and verify the listing appears.
3. **Location journey** - request location permission, test both grant and deny paths, and verify safe fallback behaviour.
4. **Trade journey** - initiate an order, complete a sandbox payment where in scope, perform QR verification according to the approved state model, and confirm completion.
5. **Chat/report journey** - open chat from an item, send a deterministic message, and access the report/block flow.
6. **Notification smoke journey** - trigger and observe one representative push-notification flow where CI/device support permits.

If a journey cannot be made deterministic or is too expensive to run on each PR, it must be assigned to the `pre` regression suite or release checklist rather than silently omitted.

Patrol tests live under `patrol_test/` and are run through `patrol test`, not `flutter test`.

7.6 Web end-to-end tests

If the React administration surface is included in the release, a browser automation tool approved by the team should cover:

- authorized administrator login;
- item/report search and review;
- permitted moderation action;
- rejection of unauthorized access;
- persistence of the moderation decision and audit information.

7.7 Manual exploratory testing and UAT

Manual testing is used for human judgment and discovery, including:

- unclear or newly changed behaviour;
- visual polish and design-guideline compliance;
- real-device camera, GPS, notification, and network behaviour;
- interrupted user journeys;
- usability and trust perceptions;
- campus pilot feedback from representative UoA users.

UAT must use predefined scenarios derived from product requirements. Feedback is logged as GitHub issues with environment, steps, expected result, actual result, severity, and evidence.

8. Critical Journey Coverage Matrix

The following matrix is the minimum expected coverage map. Detailed test cases evolve with the implementation.

Journey/capability	Unit	Widget	API/integration	Patrol/E2E	Manual/UAT	Priority
Email and Google authentication	Yes	Yes	Yes	Smoke	Yes	P0/P1
Profile and authorization	Yes	Yes	Yes	-	Yes	P0
Browse, search, filter, sort, item detail	Yes	Yes	Yes	Smoke	Yes	P1
Publish/edit/delete own item	Yes	Yes	Yes	Yes	Yes	P1
Image upload and invalid/oversized media	Yes	Yes	Yes	Targeted	Yes	P1
AI description suggestion and fallback	Yes	Yes	Contract	-	Yes	P2
Watchlist and price-change notification	Yes	Yes	Yes	Targeted	Yes	P1/P2
Chat, attachment, sanitization, report/block	Yes	Yes	Yes	Yes	Yes	P0/P1
Location permission and safe-zone suggestion	Yes	Yes	Yes	Yes	Yes	P0/P1
Trade/order state transitions	Yes	Yes	Yes	Yes	Yes	P0
Payment, failure, webhook replay, refund	Yes	Yes	Yes	Sandbox smoke	Yes	P0
Two-party QR verification and replay prevention	Yes	Yes	Yes	Yes	Yes	P0
Rating and user-credit eligibility	Yes	Yes	Yes	Targeted	Yes	P1
Abuse report lifecycle and moderation	Yes	Yes	Yes	Targeted	Yes	P0/P1
Offline/reconnect and backend failure	Yes	Yes	Yes	Targeted	Yes	P1
Dark mode and accessibility	-	Yes	-	-	Yes	P1/P2

`Targeted` means the scenario is run when the relevant native/platform capability changes and as part of `pre` regression, rather than necessarily on every small PR.

9. State and Failure Coverage

For every core journey, the test design must explicitly consider the states users rarely demonstrate manually:

- loading and delayed response;
- empty or deleted data;
- invalid or expired authentication;
- unauthorized ownership/role;
- permission denied or permanently denied;
- network timeout, offline mode, and reconnect;
- backend 4xx/5xx responses;
- vendor/API failure;

- duplicate submission, webhook replay, repeated QR scan, or double tap;
- partial completion and safe retry;
- concurrent update or stale version;
- blocked/reported user interaction;
- malformed, oversized, or unsupported media;
- user cancellation and app background/foreground transitions.

Error messages presented to users must be safe, actionable, and must not expose stack traces, secrets, internal identifiers, or sensitive data.

10. Test Environments and Data

10.1 Environment separation

Environment	Purpose	Data/services
Local	developer feedback and isolated tests	fakes, mocks, local fixtures, emulators
CI	repeatable automated verification	ephemeral services/containers, emulators, deterministic fixtures
`pre`/staging	integrated system regression and UAT	non-production project/configuration and synthetic accounts
Production	post-deployment smoke and monitoring only	production services; no destructive automated test data

10.2 Data rules

- Use synthetic identities, listings, chats, locations, reports, and transactions.
- Never use real card details; use Stripe test mode or the approved payment sandbox.
- Do not copy production personal data into tests.
- Fixtures must have stable identifiers and explicit ownership/roles.
- Tests must create or seed their own required state and clean it up or reset the environment.
- Timestamps and randomness must be controllable when they affect assertions.
- Media fixtures must include valid, unsupported, oversized, and potentially unsafe examples without containing real personal information.

10.3 Self-contained E2E setup

An E2E test must not require a developer to remember to start a service manually. The suite should start its own deterministic backend/emulators or use CI-managed containers and seeded sandbox services. Base URLs and service adapters must be configurable through dependency injection or environment-specific configuration.

11. Third-Party and Native Capability Strategy

Dependency/capability	Unit/widget approach	Integration/E2E approach
Firebase Auth / Google login	fake authentication repository and controlled states	emulator/test project and controlled test identities; native sign-in smoke only where reliable
Firestore/Firebase rules	fake repository for UI logic	emulator-based security-rule and data integration tests

Dependency/capability	Unit/widget approach	Integration/E2E approach
Node.js services/database	mock repository/gateway	isolated test database or container with deterministic seed data
Stripe/payment	fake gateway for all business states	test mode; verify webhook signatures, idempotency, failure, cancellation, and refund behaviour
FCM notifications	fake notification gateway and payload assertions	one representative device/emulator smoke; verify foreground/background handling where possible
Camera/media	fake picker/uploader	representative emulator/device flow plus validation and upload-failure tests
GPS/maps/safe zones	fake location provider and fixed coordinates	Patrol grant/deny permission flows; test-project map configuration
AI description service	deterministic fake responses, invalid output, timeout	contract/sandbox check; never make most tests depend on live model output
Social sharing	fake share adapter	native share-sheet smoke/manual verification on supported targets

The team tests KiwiShare's use of these services, not the vendors' internal implementation.

12. Non-Functional Testing

12.1 Performance

Performance profiling should cover the product risks identified in the project pitch:

- Flutter frame build/raster behaviour on product lists, image-heavy details, chat, and maps;
- memory use during repeated image selection/upload and long chat sessions;
- product-list/search latency;
- Firestore/database query and synchronization latency;
- cloud-function/API response time;
- application startup and navigation to the first useful screen.

Performance must be measured in profile/release-like mode, not inferred from debug mode. Before feature freeze, the team must record a baseline and approve measurable budgets for critical interactions. A regression beyond the approved budget requires investigation or an explicitly accepted issue.

12.2 Reliability and recovery

Verify:

- retry/backoff does not create duplicate listings, messages, orders, charges, or verification events;
- chat and QR/order state recover after restart or reconnect as specified;
- temporary backend failure produces a recoverable UI state;
- scheduled backups and restoration procedures are tested before release where applicable;
- crash reporting and backend monitoring are enabled in `pre`;
- critical errors include correlation information without leaking personal data.

12.3 Security and privacy verification

Security testing is governed by `security-owasp.md`. This strategy additionally requires automated regression coverage for:

- authentication and authorization boundaries;
- owner-only mutations;
- participant-only chat access;
- admin-only moderation access;
- input validation and sanitization;
- upload type/size restrictions;
- private/approximate location handling;
- payment webhook authenticity and replay protection;
- secrets absent from source code, logs, fixtures, and client bundles;
- safe deletion/anonymization paths where implemented.

Security-sensitive failures are treated as P0 regardless of whether the normal UI exposes the vulnerable path.

13. Device and Platform Coverage

For the coursework MVP, one supported Android emulator in CI is the minimum automated native target, consistent with course guidance. The team should also perform smoke testing on at least one representative physical Android device before major demonstrations or release.

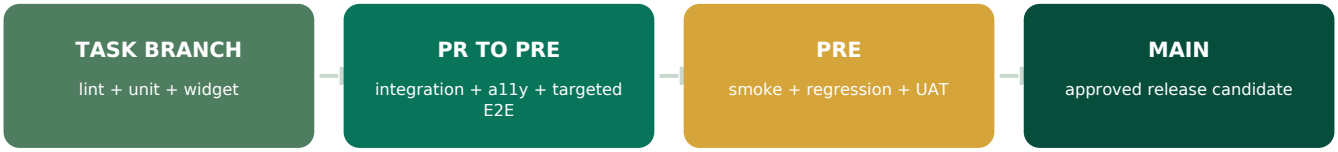
If iOS is an approved release target, the team must define a supported iOS version/device and run corresponding smoke, permission, accessibility, and payment tests. iOS support must not be claimed solely because Flutter compiles successfully.

The device matrix must consider:

- small and standard screen sizes;
- supported Android/iOS versions;
- light and dark mode;
- text scaling;
- no/slow/interrupted network;
- permission granted, denied, and previously granted states;
- foreground, background, and resumed application states.

Device-farm execution is optional for the initial coursework release and can be adopted later if the supported matrix expands.

14. Automation and Branch Quality Gates



Required checks become stronger as a change approaches release

The repository follows short-lived task branches merged into `pre` through reviewed pull requests. Tests are part of the merge contract.

14.1 Local development

Before pushing, contributors should run the relevant fast checks for the packages they changed:

```
flutter analyze
flutter test
```

When the project scripts are finalized, equivalent backend/web lint and test commands must be documented in the package manifests and root README.

14.2 Task branch push

CI should run:

- formatting/lint/static analysis;
- affected unit tests;
- affected widget/component tests;
- secret scanning and dependency checks configured by the project;
- compilation/build validation for affected packages.

14.3 Pull request into `pre`

Required checks should include:

- all task-branch checks;
- complete Flutter unit/widget suite;
- relevant backend/web test suites;
- API/service integration tests;
- accessibility tests for changed core screens;
- at least the relevant critical E2E smoke test when the change affects a P0/P1 cross-system journey.

A failed required check blocks merge. A flaky test must be fixed or formally quarantined with an owner and issue; repeatedly rerunning CI until it happens to pass is not acceptable.

14.4 `pre` regression

After merge/deployment to `pre`:

- run the integrated smoke suite;
- run scheduled or release-candidate Patrol journeys;
- perform targeted exploratory testing of the changed areas;
- verify monitoring/crash reporting;
- execute UAT scenarios for completed product requirements.

14.5 Promotion from `pre` to `main`

When the team later promotes a release, the `pre` to `main` PR must represent a tested release candidate. It requires:

- all automated required checks passing;
- full critical-journey regression passing;
- no unresolved release-blocking defects;
- approved UAT/release checklist;
- confirmed configuration, migration, monitoring, rollback, and backup readiness.

No contributor should bypass branch protection by pushing directly to `pre` or `main`.

15. Coverage and Quality Targets

The following are initial project targets and should be confirmed by the team before being enforced in CI:

- 100% of P0 acceptance criteria mapped to automated tests, with justified exceptions documented;
- at least 80% line coverage for critical business/domain logic;
- at least 70% line coverage across non-generated testable Flutter/backend code;
- every core Flutter screen covered for success, loading, empty where applicable, error, and accessibility states;
- all P0/P1 journeys represented in the coverage matrix;
- no new decrease in agreed coverage without PR justification.

Coverage percentage is a diagnostic metric, not proof of correctness. Meaningful assertions, boundary cases, and risk coverage take precedence over testing trivial code solely to increase a number. Generated code and framework boilerplate should be excluded where appropriate and documented consistently.

16. Defect Classification

Severity	Definition	Examples	Merge/release rule
S0 - Critical	security/privacy breach, financial loss, corrupt transaction state, or widespread unusability	unauthorized data access, duplicate charge, forged QR completion	immediate stop; blocks merge and release
S1 - High	primary journey blocked with no reasonable workaround	cannot authenticate, publish, pay, chat, or complete a trade	blocks affected feature merge/release

Severity	Definition	Examples	Merge/release rule
S2 - Medium	material degradation with a workaround	filter wrong in one edge case, delayed notification with recoverable state	fix or explicitly accept before release
S3 - Low	cosmetic or minor inconvenience	spacing, non-critical wording, minor animation	may be scheduled with documented priority

Every defect report should contain:

- environment/build/commit;
- preconditions and test account role;
- reproducible steps;
- expected and actual result;
- severity and affected requirement/issue;
- screenshot, recording, logs, or trace where safe;
- regression-test expectation.

17. Entry and Exit Criteria

17.1 Feature entry criteria

Implementation may begin when:

- the requirement and acceptance criteria are testable;
- interfaces/contracts and ownership are sufficiently defined;
- P0/P1 risks and required test levels are identified;
- required fakes/emulators/test accounts can be created;
- unresolved decisions that could invalidate the test design are recorded.

17.2 Feature Definition of Done

A feature is done when:

- acceptance criteria are implemented;
- the appropriate unit, widget, integration, accessibility, and E2E tests are added;
- happy paths and applicable failure states pass;
- static analysis and required CI checks pass;
- no S0/S1 defect remains;
- security/privacy impacts have been reviewed;
- documentation and test fixtures are updated;
- the PR includes test evidence and has received the required approval.

17.3 Release exit criteria

A release candidate may be promoted only when:

- all mandatory suites pass against the intended release commit;
- all P0 and P1 acceptance criteria have evidence;
- no open S0/S1 defects remain;
- accepted S2/S3 defects are documented;
- accessibility checks and critical manual checks pass;
- UAT sign-off is recorded;
- monitoring, backup, recovery, and rollback checks are complete.

18. Traceability, Naming, and Evidence

Each product requirement or GitHub issue should identify its acceptance criteria and corresponding automated tests. Recommended test identifiers are:

- `UT-<MODULE>-NNN` - unit test;
- `WT-<MODULE>-NNN` - Flutter widget/component test;
- `IT-<MODULE>-NNN` - API/service integration test;
- `E2E-<JOURNEY>-NNN` - Patrol/browser end-to-end test;
- `NFT-<AREA>-NNN` - non-functional test;
- `UAT-<JOURNEY>-NNN` - user acceptance scenario.

The PR description should report:

- affected requirements/issues;
- test levels added or updated;
- commands/suites executed;
- relevant screenshots, recordings, or performance evidence;
- known limitations or deliberately deferred cases.

19. Ownership and Maintenance

- The feature owner writes or updates tests with the implementation.
- Reviewers assess both production code and test quality.
- Module owners maintain shared fixtures, fakes, and helpers.
- Test failures are owned by the change that introduced them unless triage proves otherwise.
- Flaky, slow, or obsolete tests are engineering defects and must be tracked.
- Shared fixtures must remain minimal, explicit, and free of personal or secret data.
- This strategy is reviewed when architecture, release scope, supported platforms, or critical third-party services change.

20. Decisions Required Before CI Enforcement

The team must confirm the following before converting proposed targets into mandatory checks:

1. exact supported Android and, if applicable, iOS versions;
2. final backend/web test runners and canonical root commands;
3. Firebase emulator and isolated database strategy;
4. payment sandbox and webhook-test approach;
5. which Patrol journeys run on every PR versus scheduled/on-`pre`;
6. approved coverage thresholds and exclusions;
7. performance budgets for critical interactions;
8. whether the React administration surface is included in the MVP release;
9. ownership of test accounts, fixtures, and `pre` UAT coordination.

Until these decisions are approved, tests should still be written according to the risk and layering principles in this strategy; only the numerical and operational gates remain provisional.

21. Initial Implementation Sequence

To keep adoption achievable, implement the strategy in this order:

1. establish injectable repositories/gateways and shared deterministic fakes;
2. add unit tests for authentication, listings, orders/payment, QR verification, chat/reporting, and credit logic;
3. add widget tests for each core screen's loading, success, empty, error, interaction, and accessibility states;
4. add isolated API/security-rule/database integration tests;
5. add a small Patrol suite for publish, permission, trade/QR, and chat/report journeys;
6. wire fast checks into every task-branch push and required checks into PRs targeting `pre`;
7. establish `pre` regression, UAT, monitoring, and release evidence;
8. review test effectiveness after the first integrated milestone and adjust based on actual defects and execution time.

Appendix A - Minimum PR Testing Checklist

- ☐ Acceptance criteria are testable and linked.
- ☐ Unit tests cover new or changed logic.
- ☐ Widget/component tests cover relevant UI states.
- ☐ Accessibility checks cover changed core Flutter screens.
- ☐ API/integration tests cover changed contracts or persistence.
- ☐ P0/P1 failure and authorization paths are covered.
- ☐ Relevant Patrol/E2E journey passes, or the reason for deferral is recorded.
- ☐ No real secrets, personal data, or production endpoints appear in tests.
- ☐ Static analysis, automated tests, and affected builds pass.
- ☐ Manual evidence is attached when automation cannot verify the behaviour.

- [ ] A regression test is included for each defect fixed.

Appendix B - Minimum Core-Screen Widget Contract

For each core Flutter screen, verify as applicable:

- [ ] renders a loading state without hanging the test;
- [ ] renders expected data;
- [ ] renders an empty state;
- [ ] renders a safe error and allows retry/recovery;
- [ ] validates user input;
- [ ] enables only authorized actions;
- [ ] performs primary interaction/navigation;
- [ ] exposes meaningful semantic labels;
- [ ] meets Android/iOS tap-target and text-contrast guidelines;
- [ ] remains readable in light/dark mode and representative text scaling.

Appendix C - References

- COMPSCI 734 Lecture 07, Testing Mobile Applications, Andrew Meads, 2026.
- Five Guys, KiwiShare Elevator Pitch - Assignment 1, 2026.
- Flutter documentation, Testing Flutter apps and accessibility testing guidance.
- Patrol documentation, installation and native automation guidance.